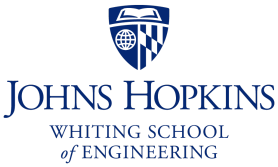
© Guven

Reinforcement Learning



Introduction

```
"I have not failed. I've just found
10,000 ways that won't work." Thomas Edison
```

Reinforcement Learning (RL) is a form of learning from **experience** through reward and penalty.
The approach is used in Artificial Intelligence, Control Theory, Psychology, Neuroscience, and Artificial Neural Networks. What makes this different from supervised learning?

- Supervised Learning: The goal is to minimize an error where the error is the difference between target output and actual output. You can compute the error because you have labeled observations (training data) that you used to build your model.
- Reinforcement Learning: The goal is to maximize the reward given by training **evaluation** (rewards and penalties). The model learns by confronting examples that don't have to exist beforehand (e.g., competing against random players in games).

Reinforcement learning also involves **policy** learning (how to map situations to actions), goal-oriented learning, and learning from interaction (i.e., games).

Features of RL

- Trial-and-Error search
- Learner decides which actions to take
- The need to balance **exploration** (of unfamiliar strategies) and **exploitation** (of tried-and-true actions)
- The whole problem of a goal-directed **agent** interacting with an uncertain environment
- Might yield a delayed reward (forgo short-term gains for greater long-term gains)

Agent-Based Learning

Consider an environment and 2 agents:



The agent continuously learns and plans. The objective is to affect the environment, which can be stochastic and uncertain. The learner employs a reward/penalty strategy to enable learning. The strength of this strategy also determines the speed of the learning. The learner retains the experience that yields gain.

Elements of RL

- Policy: what to do
- Reward: what is good
- Value: what is good because it predicts reward
- Model: what follows what

$$\text{Policy} \sqsubset \text{Reward} \sqsubset \text{Value} \sqsubset \text{Model}$$

Examples

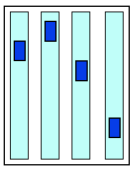
Tic-Tac-Toe Finite and small number of states; One-step look-ahead is always possible; States are completely observable $\longrightarrow$ Too easy

TD-Gammon World's best backgammon playing program

Elevator Control High-performance down-peak elevator controller (ref: Crites, R. H., Barto, A. G., "Improving elevator performance using reinforcement learning," 1996)

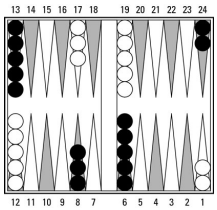
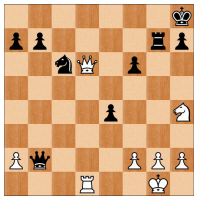
Dynamic Channel Assignment High-performance assignment of radio channels to mobile telephone calls (ref: Nie, J., Haykin, S., "A dynamic channel assignment policy through Q-learning," 1996)

Elevator Control Example

Problem	Environment	States	Actions	Rewards
	10 floors 4 elevator cars	Button states, positions Directions and motion states of cars Passengers in cars and in halls There are around $\sim 10^{22}$ number of states	Stop at, or go by, next floor	`reward` $=-1$ per time step for each person waiting

Games

Games are perfect problems for reinforcement learning.

Tic Tac Toe	Back Gammon	Chess	Go	3D Chess
				

Question: How many **states** are there for the above games?

Answer: (subjective, source: research)

- Tic tac toe: $26,830$ possible games
- Backgammon: $\sim 18,528,584,051,601,162,496$ possible positions
- Chess: $\sim 10^{53}$ possible positions
- Go: $\sim 10^{172}$ possible positions
- 3D Chess: Star Trek Tri-Dimensional chess is similar to regular chess except for the 3D partition of the board

Estimated number of atoms in the universe= 10^{80}. And AlphaGo has beaten the human Go champion recently!

Note that although these examples are all deterministic rather than stochastic, reinforcement learning also works well with stochastic games, i.e., games that have elements of chance: you could use RL to train a computer program to play Monopoly (which includes dice rolls), poker (which involves drawing cards), etc.

Evaluative Feedback

Evaluating actions $\longleftrightarrow$ **instructing** by giving correct actions

Pure evaluative feedback depends completely on the action taken
Pure instructive feedback depends not at all on the action taken

Supervised learning is instructive; optimization is evaluative.

- Associative: Inputs mapped to Outputs; learn the best output for each input
- Non-associative: *learn* (find) one best Output

n-armed bandit - Non-associative, evaluative feedback

n-Armed Bandit Problem

A slot machine is nicknamed a "one-armed bandit." A gambler with a set amount of money is trying to allocate that money between machines (which machines to play? in what order? Will he play some machine multiple times?) is said to have a "multi-armed bandit problem."

The multi-armed bandit problem generally involves a fixed, limited set of resources that must be allocated between competing or alternative choices to maximize their expected gain. These gains cannot be immediately computed, and as the learning proceeds, gains (or losses) will materialize. The problem represents the exploration versus exploitation trade-off.

Q-value: When an agent takes an action a in state s, at time t, the predicted future rewards is defined as $Q(s_t, a_t)$

(Note: The term *Q* is not from the Star Trek entity *Q*, but stands for the *Quality* of the action while learning.)

Choose repeatedly from one of n actions where each choice is called a play. After each play a_t, you get a reward r_t, where $E(r_t|a_t) = Q^*(a_t)$.

The distribution of r_t depends only on a_t. The objective is to maximize reward in the long term, e.g., over 1000 plays.

To solve the *n*-armed bandit problem, we have to **explore** a variety of actions and then **exploit** the best of them.

Exploration and Exploitation

Greedy action at t is the following:

$a_t^* = \underset{a}{\mathtt{argmax}} Q_t(a)$ ..... $(Eq.1)$

$a_t = a_t^* \implies \text{exploitation}$
$a_t \neq a_t^* \implies \text{exploration}$

Reinforcement learning requires a balance between exploration and exploitation.

Q-Learning

Based on the Bellman equation, the expected value of states and actions are stored in a table called the Q-table $Q(s,a)$. The Q-table indicates to the agent (learner) the expected reward for taking action a in state s. The values of the Q-table are updated using the equation,

$$Q(s,a) \longleftarrow Q(s,a) + \alpha \left(R(s) + \gamma \max_{a'} Q(s',a') - Q(s,a)\right) \text{.....} \quad (Eq.2)$$

We can factor out a mutiple of $(1-\alpha)$ from coefficients of the $Q(s,a)$ term and rearrange that equation:

$$Q(s,a) \longleftarrow (1-\alpha)\underbrace{Q(s,a)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \left(\underbrace{R(s)}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \underbrace{\max_{a'} Q(s',a')}_{\text{estimate of optimal future value}}\right) \text{.....} \quad (Eq.3)$$

The Q values are updated with respect to the action that maximizes the expected reward in the successor state with the equation, $\max_{a'} Q(s',a')$

$0 < \alpha < 1$ is the learning rate parameter; closer to 1 reward recent actions more
$0 < \gamma < 1$ is the discount factor, closer to 1 rewards future actions more
$R(s)$ is the reward for state s

The parameters of the update formula must be tuned so that the agent converges to the optimal policy as quickly as possible.

Learning the Nim Game

Nim is a strategy game between two players in which players remove items from three piles on each turn. For every turn, the player must remove at least one item from exactly one of the piles. There are two different versions of the game goal:
(A.) the player who clears the last pile wins
(B.) the player who clears the last pile (by removing $1 \leq$ items) loses

For the following Nim game learning example, for simplicity, in this notebook, we will consider the goal of the game to be **to clear** the last pile (game (A.) above).

We have three computer-automated players:

1. *Random* player, who moves randomly
2. *Guru* player, who moves according to the Nim solution
3. Q-learner

The following is a simple implementation of the game and the learning strategy. `st[i]` is the state for a pile `i=0,1,2`. The number of piles is hard-coded to 3, and the number of max items per pile is `ITEMS_MX` =10 in the following example. Note that pile symmetry is not handled.

Note that updates to the `qtable` should be through the strategy, but not manually as this may break the weights and weight updates to the `qtable` that are being accumulated. A consistent weighing across the entire `qtable` is important.

In [1]:
```
import numpy as np
from random import randint, choice

# The number of piles is 3


# max number of items per pile
ITEMS_MX = 10

# Initialize starting position
def init_game()->list:
    return [randint(1,ITEMS_MX), randint(1,ITEMS_MX), randint(1,ITEMS_MX)]

# Based on X-oring the item counts in piles - mathematical solution
def nim_guru(_st:list)->(int,int):
    xored = _st[0] ^ _st[1] ^ _st[2]
    if xored == 0:
        return nim_random(_st)
    for pile in range(3):
        s = _st[pile] ^ xored
        if s <= _st[pile]:
            return _st[pile]-s, pile

# Random Nim player
def nim_random(_st:list)->(int,int):
    pile = choice([i for i in range(3) if _st[i]>0])  # find the non-empty
    return randint(1, _st[pile]), pile  # random move
```

In the following cell, the Q-learner is presented. The function makes a move according to the *learned* Q-table. The Q-table format is a matrix of type `float`

and has dimensions `(number_of_states, possible_moves)` which covers all possible states and actions there can be for the Nim game.

The states are simply a list of the number of items in the first pile, the number of items in the second pile, and the number of items in the third pile.
e.g., `st = [10,10,10]` can be a starting state

The move or action is the tuple of the number of items to remove and the pile ID.
e.g., `a = (2,0)` can be an action

For ease of programming and assuring the Q-table does not contain illegal moves (e.g., move (0,0) - not removing an item from a pile), the following table indexing is used:
`index = pile*ITEMS_MX + move-1` $\longleftrightarrow$ `move, pile = index%ITEMS_MX+1, index//ITEMS_MX`

Note that indexing is 0-based.

The first index part is the pile, i.e., 0, 1, 2. The second index part uses a numbering system based on the number of max items per pile, i.e., using max items of 10. Examples,
index 2 $\longrightarrow$ move 3, pile 0
index 11 $\longrightarrow$ move 2, pile 1
move 10, pile 2 $\longrightarrow$ index 29

Note: In RL, the model developer designs the states and the actions (i.e., game move).

In [2]:
```
def nim_qlearner(_st:list)->(int,int):
    global qtable
    # pick the best rewarding move, equation 1
    a = np.argmax(qtable[_st[0], _st[1], _st[2]])  # exploitation
    # index is based on move, pile
    move, pile = a%ITEMS_MX+1, a//ITEMS_MX
    # check if qtable has generated a random but game illegal move - we ha
    if move <= 0 or _st[pile] < move:
        move, pile = nim_random(_st)  # exploration
    return move, pile  # action
```

We will use the engines `nim_guru`, `nim_random` and `nim_qlearner` to play games, as below.

In [3]:
```
Engines = {'Random':nim_random, 'Guru':nim_guru, 'Qlearner':nim_qlearner}

def game(_a:str, _b:str):
    state, side = init_game(), 'A'
    while True:
```

```
        engine = Engines[_a] if side == 'A' else Engines[_b]
        move, pile = engine(state)
        # print(state, move, pile)  # debug purposes
        state[pile] -= move
        if state == [0, 0, 0]:  # game ends
            return side  # winning side
        side = 'B' if side == 'A' else 'A'  # switch sides

def play_games(_n:int, _a:str, _b:str)->(int,int):
    from collections import defaultdict
    wins = defaultdict(int)
    for _ in range(_n):
        wins[game(_a, _b)] += 1
    # info
    print(f"{_n} games, {_a:>8s}{wins['A']:5d}  {_b:>8s}{wins['B']:5d}")
    return wins['A'], wins['B']
```

In [4]:
```
# Play games
play_games(1000, 'Random', 'Random')
play_games(1000, 'Guru', 'Random')
play_games(1000, 'Random', 'Guru')
play_games(1000, 'Guru', 'Guru') ;
```

```
1000 games,   Random  509    Random  491
1000 games,     Guru  999    Random    1
1000 games,   Random    7      Guru  993
1000 games,     Guru  933      Guru   67
```

Starting the Nim game first has an advantage for the Guru player.

(Note that with its perfect mathematical solution, the Guru wins *close to but not quite* 100% of games when it moves first vs. Guru. This is because, although a specific game of Nim is deterministic, each game of Nim starts with a random number (from 1 to 10) of elements in each pile. It looks like perfect play lets Player One force a win in 94% of configurations with these parameters of `piles=3`, `ITEMS_MX=10`.)

Now let's see how the Q-learner actually learns by the Q-learning algorithm.

Alpha α is set to 1 since the Nim game is deterministic, and every winning state must be learned.
For stochastic environments, where events happen randomly or cannot be predicted, alpha is set to less than 1.

In [5]:
```
qtable, Alpha, Gamma, Reward = None, 1.0, 0.8, 100.0

# learn from _n games, randomly played to explore the possible states
def nim_qlearn(_n:int):
    global qtable
    # based on max items per pile
    qtable = np.zeros((ITEMS_MX+1, ITEMS_MX+1, ITEMS_MX+1, ITEMS_MX*3), dt
    # play _n games
```

```
    for _ in range(_n):
        # first state is starting position
        st1 = init_game()
        while True:  # while game not finished
            # make a random move - exploration
            move, pile = nim_random(st1)
            st2 = list(st1)
            # make the move
            st2[pile] -= move  # --> last move I made
            if st2 == [0, 0, 0]:  # game ends
                qtable_update(Reward, st1, move, pile, 0)  # I won
                break  # new game

            qtable_update(0, st1, move, pile, np.max(qtable[st2[0], st2[1]

            # Switch sides for play and learning
            st1 = st2

# Equation 3 - update the qtable
def qtable_update(r:float, _st1:list, move:int, pile:int, q_future_best:fl
    a = pile*ITEMS_MX+move-1
    qtable[_st1[0], _st1[1], _st1[2], a] = Alpha * (r + Gamma * q_future_b
```

Try learning by 1000 games only.

In [6]:
```
%%time

nim_qlearn(1000)
```

```
CPU times: total: 31.2 ms
Wall time: 23.1 ms
```

In [7]:
```
# Play games
play_games(1000, 'Qlearner', 'Random')
play_games(1000, 'Random', 'Qlearner')

play_games(1000, 'Random', 'Random') ;
```

```
1000 games, Qlearner  735    Random  265
1000 games,   Random  279  Qlearner  721
1000 games,   Random  521    Random  479
```

As can be seen above our Q-learner learns 'something' and can beat the random player.

Now let's see if the amount of learning has any effect?

In [8]:
```
%%time

# See the training size effect
n_train = (3, 10, 100, 1000, 10000, 50000, 100000)
Wins = []
for n in n_train:
    nim_qlearn(n)
```

```
    wins_a, wins_b = play_games(1000, 'Qlearner', 'Random')
    Wins += [wins_a/(wins_a+wins_b)]
```

```
1000 games, Qlearner  486    Random  514
1000 games, Qlearner  569    Random  431
1000 games, Qlearner  685    Random  315
1000 games, Qlearner  717    Random  283
1000 games, Qlearner  690    Random  310
1000 games, Qlearner  719    Random  281
1000 games, Qlearner  725    Random  275
CPU times: total: 4.34 s
Wall time: 4.34 s
```

In [9]:
```
# Check the ratio of wins wrt to size of the reinforcement model training
print(Wins)
```

```
[0.486, 0.569, 0.685, 0.717, 0.69, 0.719, 0.725]
```

Learning from even 10 games works, but after 100 games, our learner actually converges.

Question: It looks like our learner does not learn everything that could be learned. How would you modify and update the above code to make the Q-learner learn the Nim game and be able to beat the Guru player when it starts as the first player? We will look at this more in the Assignment.

Note: The following code helps to debug your Q-learner. Different strategies can be examined by `qtable` comparison.

In [10]:
```
# Function to print the entire set of states
def qtable_log(_fn:str):
    with open(_fn, 'w') as fout:
        s = 'state'
        for a in range(ITEMS_MX*3):
            move, pile = a%ITEMS_MX+1, a//ITEMS_MX
            s += ',%02d_%01d' % (move,pile)
        print(s, file=fout)
        for i, j, k in [(i,j,k) for i in range(ITEMS_MX+1) for j in range(
            s = '%02d_%02d_%02d' % (i,j,k)
            for a in range(ITEMS_MX*3):
                r = qtable[i, j, k, a]
                s += ',%.1f' % r
            print(s, file=fout)

qtable_log('qtable_debug.txt')
```

References

1. Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.
2. Watkins, Christopher JCH, and Peter Dayan. "Q-learning." Machine learning 8.3 (1992): 279-292.
3. Järleberg, Erik. "Reinforcement learning on the combinatorial game of Nim." (2011).

Exercises

Exercise 1. Examine the Q-table and find columns that are all zero. What are they and why do they exist?

Exercise 2. Set the $\alpha = 1$ and $\gamma = 0$. Observe the behavior of the learner and report it.

In [11]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
